PROIEKTUAREN DISEINU PATROIAK

Egileak:

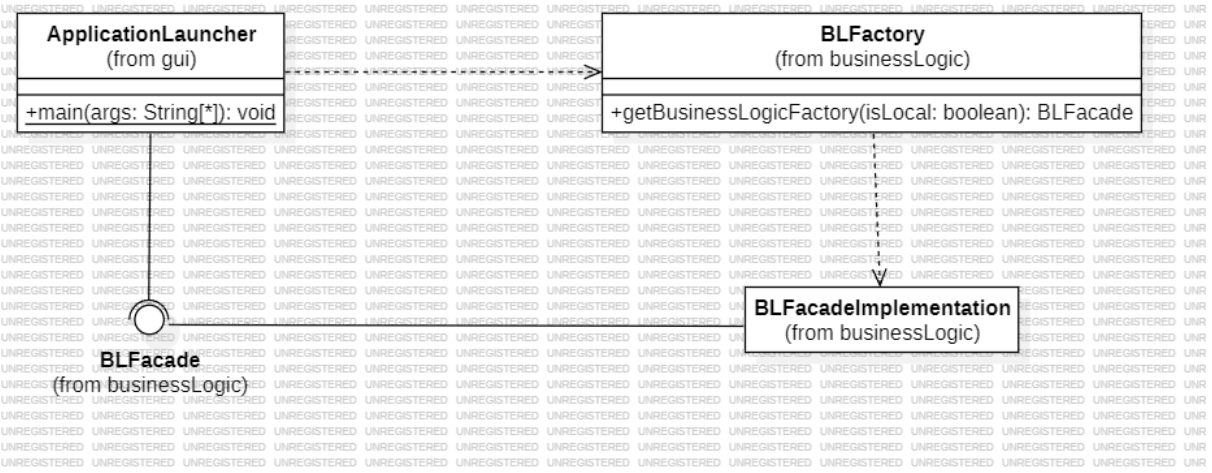
- Danel Larreategui
- Imanol Egaña
- Markel Sanchez

Aurkibidea

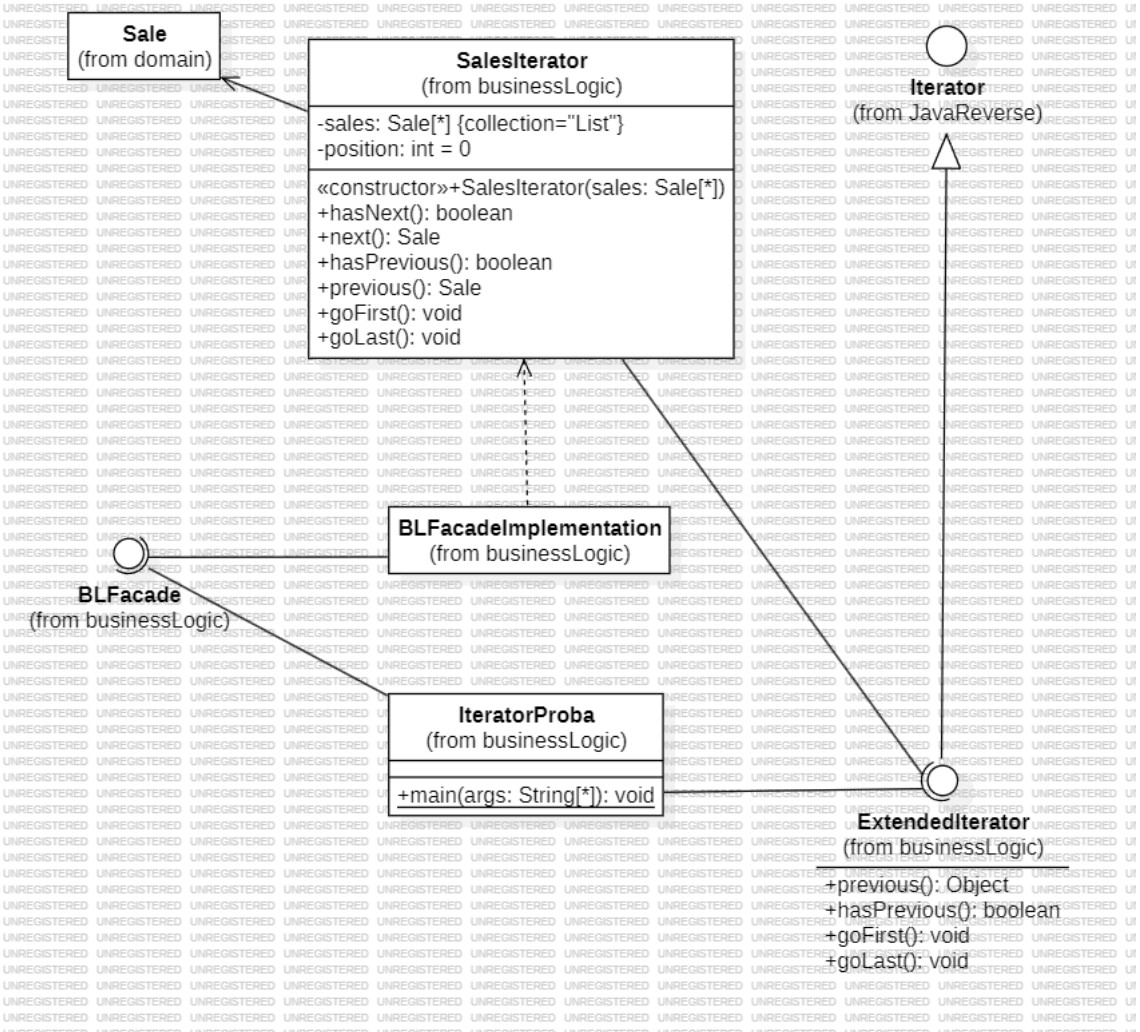
1. UML diagrama hedatua.....2
1.1. Factory..... 2
1.2. Iterator.....2
1.3. Adapter.....3
2. Aldatutako kodea..... 4
2.1. Factory..... 4
2.2. Iterator.....6
2.3. Adapter.....9
3. Iterator eta Adapter patroientzako, exekuzioaren irudia..... 12
3.1. Adapter patroiaren exekuzioaren irudia.....12
3.2. Iterator patroiaren exekuzioaren irudia.....12

1. UML diagrama hedatua

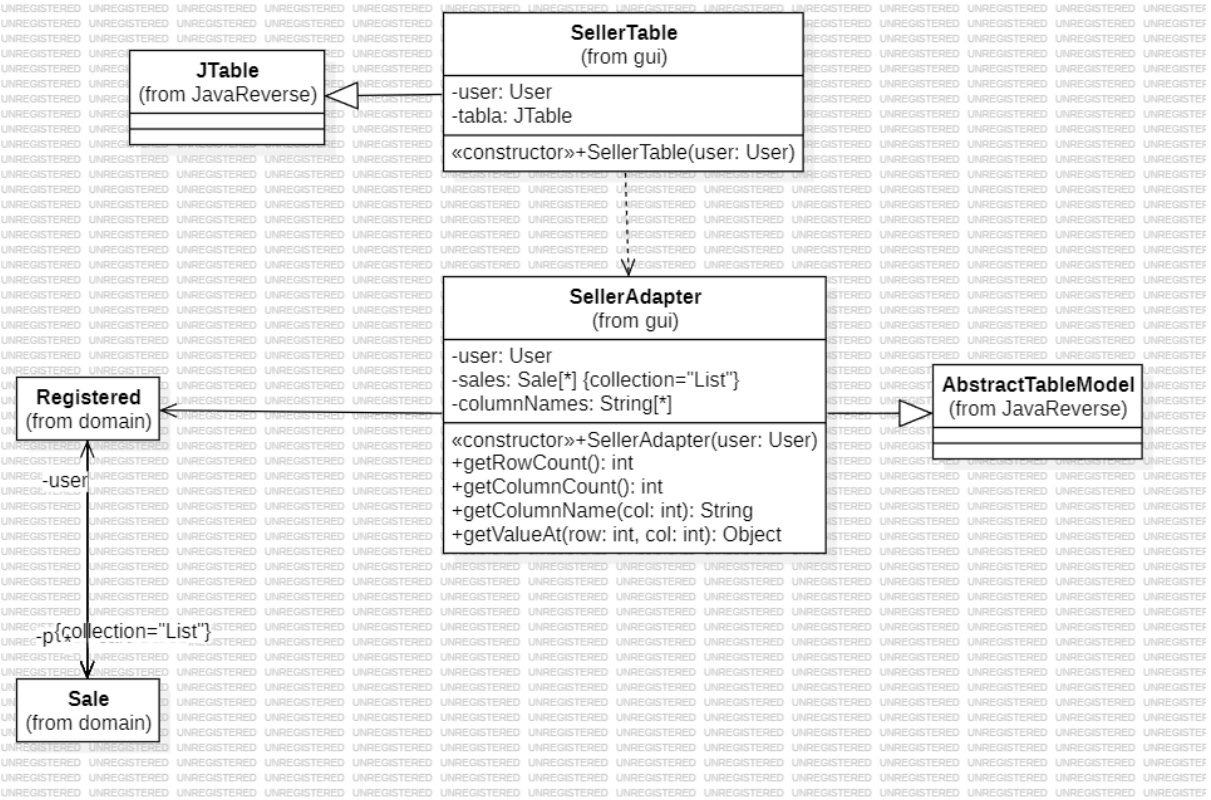
1.1. Factory



1.2. Iterator



1.3. Adapter


SellerTable
(from gui)
-user: User
-tabla: JTable
«constructor»+SellerTable(user: User)
JTable
(from JavaReverse)
SellerAdapter
(from gui)
-user: User
-sales: Sale[*] {collection="List"}
-columnNames: String[*]
«constructor»+SellerAdapter(user: User)
+getRowCount(): int
+getColumnCount(): int
+getColumnName(col: int): String
+getValueAt(row: int, col: int): Object
AbstractTableModel
(from JavaReverse)
Registered
(from domain)
-user
-p {collection="List"}
Sale
(from domain)

2. Aldatutako kodea

2.1. Factory

BLFactory.java

```java
package businessLogic;

import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import configuration.ConfigXML;

public class BLFactory {

    public BLFacade getBusinessLogicFactory(boolean isLocal) throws Exception {
        ConfigXML c = ConfigXML.getInstance();

        if (isLocal) {
            return new BLFacadeImplementation();
        } else {
            String serviceName = "http://" + c.getBusinessLogicNode() + ":"
                                 + c.getBusinessLogicPort() + "/ws/"
                                 + c.getBusinessLogicName() + "?wsdl";
            URL url = new URL(serviceName);
            QName qname = new QName("http://businessLogic/",
"BLFacadeImplementationService");
            Service service = Service.create(url, qname);
            return service.getPort(BLFacade.class);
        }
    }
}
```

- **public BLFacade getBusinessLogicFactory(boolean isLocal):** Negozio-logikaren instantzia lokala edo urrunekoa sortzeaz arduratzen den metodo nagusia da. Honela, objektuak sortzeko ardura leku bakarrean zentralizatzen da (Creator rola).
- **return new BLFacadeImplementation(da); eta return service.getPort(BLFacade.class);** Metodoak beti BLFacade interfaze bera itzultzen dio bezeroari, atzean zein inplementazio zehatz (ConcreteProduct) dagoen ezkutatuz.

ApplicationLauncher.java

```java
package gui;

import java.awt.Color;
import java.util.Locale;
import javax.swing.UIManager;
import businessLogic.BLFacade;
import businessLogic.BLFactory;
import configuration.ConfigXML;

public class ApplicationLauncher {

    public static void main(String[] args) {
        ConfigXML c = ConfigXML.getInstance();
        System.out.println(c.getLocale());
        Locale.setDefault(new Locale(c.getLocale()));
        System.out.println("Locale: " + Locale.getDefault());

        MainGUI a = new MainGUI("");
        a.setVisible(true);

        try {
           UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");

            boolean isLocal = c.isBusinessLogicLocal();
            BLFactory factory = new BLFactory();
            BLFacade appFacadeInterface =
factory.getBusinessLogicFactory(isLocal);

            MainGUI.setBussinessLogic(appFacadeInterface);

        } catch (Exception e) {
            a.jLabelSelectOption.setText("Error: " + e.toString());
            a.jLabelSelectOption.setForeground(Color.RED);
            System.out.println("Error in ApplicationLauncher: " +
e.toString());
        }
    }
}
```

- **BLFacade appFacadeInterface = new BLFactory().getBusinessLogicFactory(isLocal);:** Hasierako kodean main metodoan zegoen if/else egitura luzea ezabatu da. Orain, aurkezpen-mailak faktoriari eskatzen dio instantzia. Aldaketa honek klaseen arteko akoplamendua nabarmen murrizten du, kodea garbiago utziz.

2.2. Iterator

ExtendedIteratorSales.java

```java
public interface ExtendedIterator<Object>      extends Iterator<Object> {
	//return     the    actual element      and    go     to     the
previous
	public Object previous();
	//true i f ther is a previous element
	public boolean hasPrevious();
	//It is placed      in     the    first  element
	public void goFirst();
	//It   is     placed in     the    last   element
	public void goLast();
}
```

BLFacadeImplementation.java

```java
@WebMethod
	public ExtendedIterator<Sale> getSalesIterator(String desc) {
		List<Sale> salesList = this.getSales(desc);
		return new SalesIterator(salesList);
	}
```

- Datu-basetik ohiko zerrenda (List<Sale>) lortzen du, baina bezeroari (GUI-ari) zuzenean itzuli beharrean, gure ExtendedIteratorSales klasean biltzen du iterazio aurreratua baimenduz.

SalesIterator:

```java
public class SalesIterator implements ExtendedIterator<Sale> {

	private List<Sale> sales;
	private int position = 0;

	public SalesIterator(List<Sale> sales) {
		this.sales = sales;
	}
	public boolean hasNext() {
		return position < sales.size();
	}
```

```
	public Sale next() {
		if (!hasNext()) {
			throw new NoSuchElementException("Ez daude produktu
gehiago!");
		}
		Sale sale = sales.get(position);
		position++;
		return sale;
	}
	@Override
	public boolean hasPrevious() {
		return position > 0;
	}
	@Override
	public Sale previous() {
		if (!hasPrevious()) {
			throw new NoSuchElementException("Ez daude produktu
gehiago!");
		}
		position--;
		return sales.get(position);
	}
	@Override
	public void goFirst() {
		position = 0;
	}

	@Override
	public void goLast() {
		position = sales.size();
	}
}
```

- **private int position:** Iteratzailearen uneko egoera (zerrendan zein indizetan gauden) gordetzen duen aldagaia da.
- **public Sale previous():** Indizea txikituz zerrendan atzeraka joatea ahalbidetzen du.
- **public void goLast():** Kurtsorea zuzenean zerrendaren azken elementuan kokatzen du, uneko tamainari 1 kenduta, atzeranzko ibilbidea hasi ahal izateko.

IteratorProba:

```java
public class IteratorProba {
	public static void main(String[] args) {
		boolean isLocal = true;
		BLFacade blFacade;
		try {
			blFacade = new
BLFactory().getBusinessLogicFactory(isLocal);

			ExtendedIterator<Sale> it = blFacade.getSalesIterator("a");
			Sale s;
			System.out.println("_____________________");
			System.out.println("FROM   LAST   TO    FIRST");
			it.goLast(); // Go to last element
			while (it.hasPrevious()) {
				s = it.previous();
				System.out.println(s);
			}
			System.out.println();
			System.out.println("_____________________");
			System.out.println("FROM   FIRST  TO    LAST");
			it.goFirst(); // Go to first element
			while (it.hasNext()) {
				s = it.next();
				System.out.println(s);
			}
		} catch (Exception e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}
	}
}
```

2.3. Adapter

AdapterLauncher:

```java
package gui;

import businessLogic.BLFacade;
import businessLogic.BLFactory;
import domain.User;

public class AdapterLauncher {

    public static void main(String[] args) {
        try {
            boolean isLocal = true;
            BLFacade blFacade = new
BLFactory().getBusinessLogicFactory(isLocal);

            User u = blFacade.isLogged("seller1@gmail.com", "aurrera");

            if (u != null) {
                SellerTable st = new SellerTable(u);
                st.setVisible(true);
            } else {
                System.out.println("Error: Ez da aurkitu user-a edo pasahitz
okerra.");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

SellerAdapter:

```java
package gui;

import java.util.List;
import javax.swing.table.AbstractTableModel;
import domain.Sale;
import domain.User;
import domain.Registered;

public class SellerAdapter extends AbstractTableModel {
    private User user;
```

```
    private List<Sale> sales;
    private String[] columnNames = new String[] { "title", "description",
"price" };

    public SellerAdapter(User user) {
        this.user = user;

        if (this.user != null && this.user instanceof Registered) {
            this.sales = ((Registered) this.user).getSales();
        }
    }

    @Override
    public int getRowCount() {
        if (sales == null) return 0;
        return sales.size();
    }

    @Override
    public int getColumnCount() {
        return columnNames.length;
    }

    @Override
    public String getColumnName(int col) {
        return columnNames[col];
    }

    @Override
    public Object getValueAt(int row, int col) {
        if (sales == null || row >= sales.size()) return null;

        Sale sale = sales.get(row);
        switch (col) {
            case 0:
                return sale.getTitle();
            case 1:
                return sale.getDescription();
            case 2:
                return sale.getPrice();
            default:
                return null;
        }
    }
}
```

- **public class SellerAdapter extends AbstractTableModel:** Lerro hau funtsezkoa da. Patroiaren "Target" edo helburua da. AbstractTableModel hedatuz, Swing-eko JTable osagaiak gure klasea taula-eredu estandar gisa ulertuko du.

- **this.sales = ((Registered) this.user).getSales():** Hemen gertatzen da egokitzapen edo konbertsio nagusia. "Adaptee" (User) objektua jasotzen da, Registered motara cast-a egiten da eskaerak/salmentak lortzeko, eta egokitzailearen memorian gordetzen da.

- **public Object getValueAt(int row, int col):** Taulako ilara eta zutabe zehatz bat marraztu behar denean, metodo honek gure Sale objektuaren atributuekin mapatzen du (0 zutabea getTitle(), 1 zutabea getDescription(), etab.), informazioa interfazeak eskatzen duen formatura itzuliz.

SellerTable:

```java
package gui;

import java.awt.BorderLayout;
import java.awt.Dimension;
import javax.swing.JFrame;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import domain.User;

public class SellerTable extends JFrame {
    private User user;
    private JTable tabla;

    public SellerTable(User user) {
        super(user.getEmail() + "'s sales ");
        this.setBounds(100, 100, 700, 200);
        this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.user = user;
        SellerAdapter adapt = new SellerAdapter(user);
        tabla = new JTable(adapt);
        tabla.setPreferredScrollableViewportSize(new Dimension(500, 70));
        JScrollPane scrollPane = new JScrollPane(tabla);
        getContentPane().add(scrollPane, BorderLayout.CENTER);
    }
}
```

- SellerAdapter adapt = new SellerAdapter(user); tabla = new JTable(adapt);: Datuak egokitzen dituen Adapter objektua sortzen da eta JTable osagaiaren eraikitzaileari pasatzen zaio. JTable-ak ez daki atzean User edo Sale motako datuak daudenik; berak AbstractTableModel arrunt bat bezala tratatzen du.

3. Iterator eta Adapter patroientzako, exekuzioaren irudia

3.1. Adapter patroiaren exekuzioaren irudia



3.2. Iterator patroiaren exekuzioaren irudia

